

Sécurisation des communications

Objectifs :

- ✓ Savoir décrire les principes de chiffrement symétrique (clé partagée) et asymétrique (clé privée/clé publique) ;
- ✓ Savoir décrire l'échange d'une clé symétrique en utilisant un protocole asymétrique pour sécuriser une communication HTTPS.

I – Un peu de vocabulaire

- **Coder** : représenter de l'information par un ensemble de signes prédéfinis. On utilise parfois le verbe *encoder*.
- **Décoder** : interpréter un ensemble de signes pour extraire l'information qu'ils représentent.
- **Chiffrer** : rendre une suite de symboles incompréhensible au moyen d'une clé de chiffrement.
- **Déchiffrer/décrypter** : retrouver la suite de symboles originale à partir du message chiffré. On utilise le terme *déchiffrer* lorsque l'on utilise une *clé de déchiffrement* pour récupérer le texte initial et le terme *décrypter* lorsque l'on arrive à déterminer le message original sans utiliser de clé.

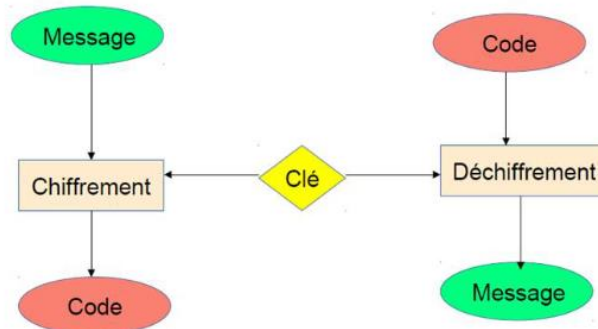
Coder et décoder s'utilisent lorsqu'il n'y a pas de secret. Par exemple, on parle de « codage en complément à deux » des entiers. C'est juste une façon de représenter les entiers positifs ou négatifs par une suite de bits. N'importe qui peut décoder la suite de bits pour déterminer l'entier. Le terme « coder » est aussi utilisé de façon informelle comme synonyme de « programmer ». Par exemple : « j'ai codé cette application ». L'anglicisme « crypter » est à proscrire : on utilisera « chiffrer ».

II – Chiffrement symétrique

Une première technique lorsque l'on souhaite chiffrer un message est d'utiliser une méthode de chiffrement symétrique. Formellement, une telle méthode est donnée par deux fonctions :

- $c(m,k)$ est une fonction de chiffrement qui prend en arguments un message en clair m (une chaîne de caractères par exemple) et une clé de chiffrement k (qui peut être une chaîne de caractères, un nombre, etc.). Elle produit en sortie une chaîne de caractères chiffrée appelée s .
- $d(s,k)$ est la fonction de déchiffrement. Elle prend en arguments un message chiffré s , une clé de déchiffrement k , et renvoie le message en clair m .

Le terme symétrique vient du fait que la même clé est utilisée pour chiffrer et déchiffrer le message.



1) Le chiffrement de César

Un exemple simple de chiffrement symétrique est le chiffrement par décalage, encore appelé chiffrement de César (de l'empereur romain Jules César qui utilisait cette technique pour ses correspondances militaires).

La méthode consiste à choisir un entier n et à décaler chaque lettre du message initial de n lettres dans l'alphabet, en recommençant à « A » si le décalage fait dépasser « Z ». Ici, l'entier n constitue la clé de chiffrement.

Par exemple, en utilisant un décalage de 5 lettres, le message

L INFORMATIQUE C EST SUPER devient Q NSKTWRFYNVZJ H JXY XZUJW

La fonction de déchiffrement consiste à prendre un message chiffré et à décaler chaque lettre de n positions vers l'arrière.

Sécurisation des communications

Exercices :

- Notebook *Cryptographie_sujet.ipynb*
- Session 2021 – Exercice n°1 du sujet 06 (21-NSIJ1G11)

2) La méthode XOR

Une méthode de chiffrement un peu moins naïve est le chiffrement par XOR (ou exclusif). Celle-ci repose sur l'utilisation de l'opérateur binaire *ou exclusif* noté $\oplus$. Etant donné un message m (par exemple « L'INFORMATIQUE C'EST SUPER ») et une clé chiffrement k (par exemple, « NSI »). On recopie plusieurs fois la clé de façon à obtenir une chaîne de la même longueur que le message :

L'INFORMATIQUE C'EST SUPER
NSINSINSINSINSINSINSINSINS

Chaque caractère du message et de la clé augmentée est ensuite converti en nombre.
Par exemple, son code Unicode :

76 39 73 78 70 79 82 77 65 **84** 73 ...
78 83 73 78 83 73 78 83 73 **78** 83 ...

On effectue ensuite l'opération *ou exclusif* entre chaque nombre du message et de la clé. Nous détaillons, ci-dessous, entre les nombres 84 (lettre « T ») et 78 (lettre « N ») :

$$\begin{array}{cccccccc} & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & (84) \\ \oplus & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & (78) \\ \hline & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & (26) \end{array}$$

On rappelle que l'opération *ou exclusif* entre deux bits renvoie 0 si les deux bits sont égaux et 1 s'ils sont différents.
Le message chiffré sera donc :

2 116 0 0 21 6 28 30 8 26 26 ...

Une propriété intéressante de l'opérateur *ou exclusif* est qu'il est réversible :

- Si $A \oplus B = C$, alors $A \oplus C = B$ et $B \oplus C = A$

Par conséquent, le message chiffré ci-dessus peut être déchiffré en réexécutant l'opération *ou exclusif*, nombre à nombre, avec les entiers formant la clé étendue.

De plus, l'opérateur *ou exclusif* est l'une des opérations de base implémentée par le matériel dans l'unité arithmétique et logique du micro-processeur. Ces caractéristiques font que l'opérateur *ou exclusif* est une brique de base couramment utilisée dans les algorithmes de chiffrement modernes (même si, utilisé naïvement comme nous l'avons fait ci-dessus, il n'apporte pas une grande sécurité).

Parmi les algorithmes de chiffrement symétrique, on peut citer AES (Advanced Encryption Standard, standard de chiffrement avancé). Bien que très complexe, cet algorithme repose sur des principes de chiffrement similaires à ceux décrits plus haut pour l'opérateur $\oplus$.

Exercices :

- Session 2021 – Exercice n°2 du sujet 02
- Session 2021 – Exercice n°3 du sujet 07 (21-NSIJ2G11)

Sécurisation des communications

III – Chiffrement asymétrique

1) Principe du chiffrement asymétrique

Malgré les nombreux avantages du chiffrement symétrique, ce dernier possède un défaut important.

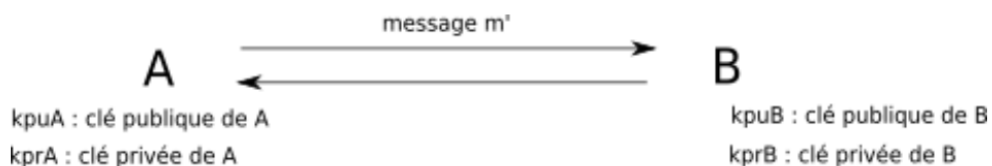
En effet, si deux personnes veulent établir un canal de communication sûr en utilisant une méthode de chiffrement symétrique, les deux personnes doivent d'abord se mettre d'accord sur la clé à utiliser (et évidemment sur l'algorithme lui-même). Or, les deux participants sont justement dans la situation où ils ne peuvent *pas encore* communiquer de façon sûre. Ils vont donc avoir un problème pour se mettre d'accord sur la clé :

- soit ils échangent la clé par un moyen de communication non sûr (par e-mail, par courrier, etc.) mais dans ce cas, un attaquant pourrait alors s'emparer de la clé et compromettre la sûreté des futures communications chiffrées avec cette clé ;
- soit ils échangent la clé par un moyen « non pratique », par exemple en stockant une clé générée sur un support de stockage et en se rencontrant physiquement pour se donner la clé de chiffrement...

Pour résoudre ce problème, diverses techniques ont été développées dès les années 1970, d'abord de manière privée, en particulier par les services secrets britanniques et américains, puis dans la recherche académique publique. Ces techniques reposent sur la cryptographie *asymétrique*, encore appelée *cryptographie à clé publique*.

Dans le cas du chiffrement asymétrique A et B n'ont pas besoin de partager une "clé secrète" :

- A possède une "clé privée" que l'on notera k_{prA} et une "clé publique" que l'on notera k_{puA} .
En aucun cas A, ne devra divulguer sa clé privée à quiconque, elle devra rester strictement secrète.
En revanche, sa clé publique pourra être connue de tout le monde sans aucun problème.
- B possède une "clé privée" que l'on notera k_{prB} et une "clé publique" que l'on notera k_{puB} .
En aucun cas B ne devra divulguer sa clé privée à quiconque, elle devra rester strictement secrète.
En revanche, sa clé publique pourra être connue de tout le monde sans aucun problème.



Un exemple classique : la méthode de Diffie-Hellman

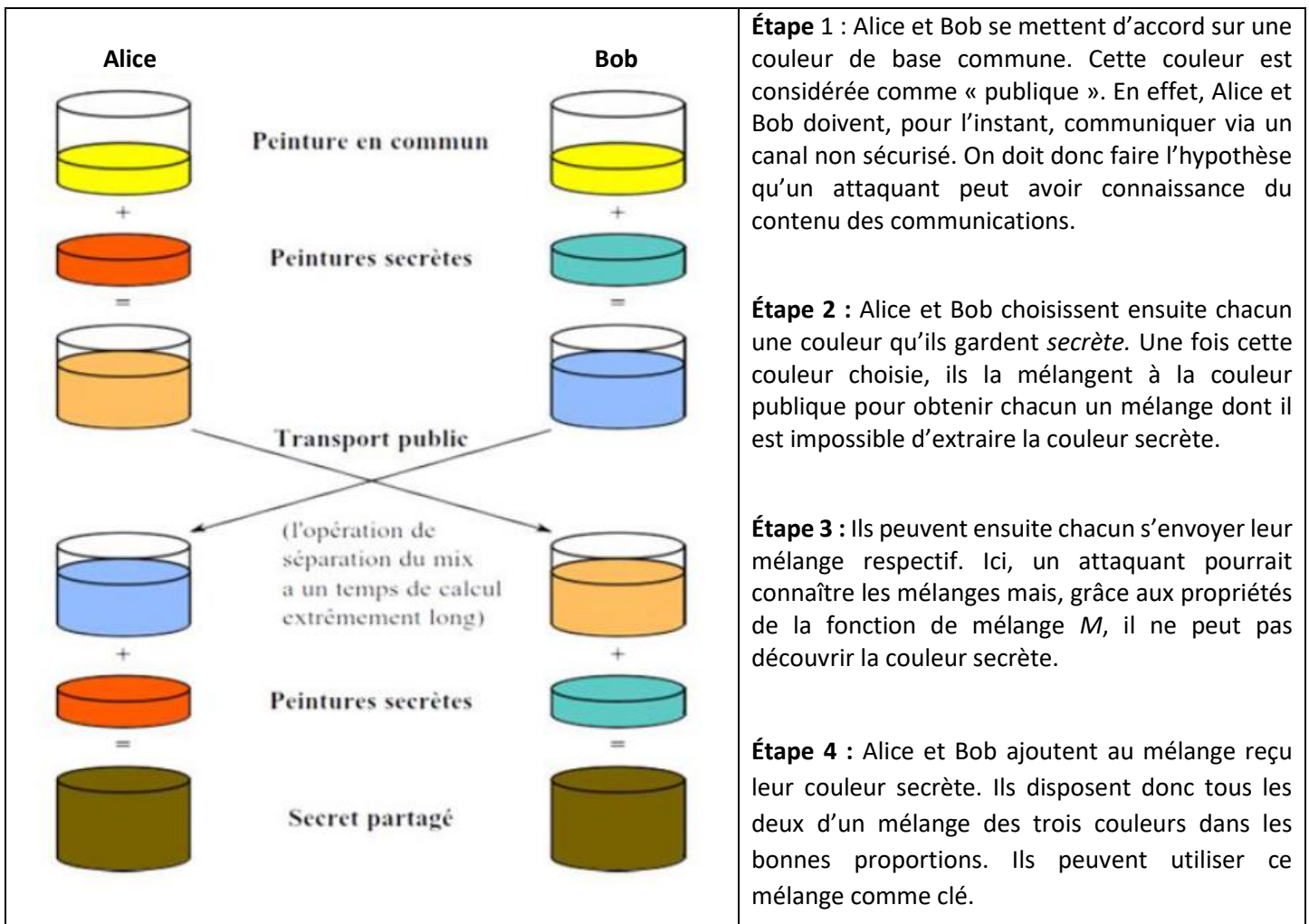
Inventé en 1976 par Whitfield Diffie (1944 -) et Martin Hellman (1945 -), deux cryptologues américains, le chiffrement asymétrique vient résoudre l'inconvénient essentiel du chiffrement symétrique : le nécessaire partage d'un secret (la clé) avant l'établissement de la communication sécurisée. Le principe est le suivant.

Soit une fonction mathématique M (pour « mélange ») prenant en arguments deux entiers et possédant les propriétés suivantes :

1. La fonction M est connue (on connaît sa formule ou l'algorithme qui permet de la calculer).
2. Si on connaît $M(x, y)$ et x alors il est « difficile » de retrouver y (clé secrète). Par difficile, on souhaite que la seule façon dont on dispose pour trouver un y tel que $M(x, y)$ ait une valeur donnée est d'essayer tous les entiers y possibles.
3. Pour tous entiers x, y et z , on a $M(M(x, y), z) = M(M(x, z), y)$.

Sécurisation des communications

Une analogie couramment utilisée est celle des pots de peintures : supposons que l'on ait à notre disposition un très grand nombre de couleurs de peintures différentes.



Notre fonction mélange M possède bien les trois propriétés ci-dessus :

- La fonction M prélève 10 cl de deux couleurs et les mélange (pour obtenir 20 cl de peinture) : nous connaissons M et tout un chacun (muni d'un verre doseur) peut la reproduire (propriété 1).
- Etant données deux couleurs (par exemple jaune et bleu), si on les mélange on obtient 20 cl d'une couleur bien particulière. Il semble a priori impossible, en ayant à notre disposition la couleur du mélange et la référence exacte du jaune, de deviner quel bleu a été utilisé car, pour de la peinture, il n'existe pas d'opération de « soustraction » qui permettrait de retirer le jaune (propriété 2).
- Enfin, si on mélange d'abord le jaune et le bleu puis le rouge on obtient au final 30 cl d'un certain marron. On obtient 30 cl du même marron si on mélange d'abord le jaune et le rouge, puis qu'on ajoute le bleu (prop. 3). Une remarque importante est que, même si on réunit les deux mélanges de l'étape 3, on obtiendra 40 cl d'un marron *différent* (car la proportion de jaune est plus importante).

L'échange de clés de Diffie-Hellman permet ainsi aux deux protagonistes Alice et Bob de construire une clé de chiffrement secrète commune. Celle-ci permettra de sécuriser leurs prochains échanges, y compris en utilisant un chiffrement symétrique avec cette clé.

Cette idée valut en 2015 aux deux auteurs le prix Turing.

Sécurisation des communications

En pratique, on n'échange pas des pots de peinture...

<https://dane.web.ac-grenoble.fr/outils-numeriques-0/decoupeur-de-video#fNwVKNx7u6w#0#0#0>

Source :

Vidéo intitulée « *Le chiffrement asymétrique - Diffie-Hellman* » extraite de la chaîne de Mickaël Dupont, enseignant-chercheur à l'USMB Savoie-Mont-Blanc

Un autre exemple classique : le système RSA

Le chiffrement RSA est un algorithme de cryptographie asymétrique, très utilisé dans le commerce électronique, et plus généralement pour échanger des données confidentielles sur Internet. Il doit son nom à ses trois inventeurs : Ron **R**ivest (1947-), cryptologue américain, Adi **S**hamir (1952-), cryptologue israélien, et Len **A**dleman (1945-), cryptologue américain.

Le système RSA est basé sur la théorie des nombres premiers, et plus particulièrement sur l'arithmétique modulaire. Il repose sur des théorèmes mathématiques qui vont bien au-delà du programme de terminale.

Cependant, il faut savoir que le chiffrement RSA repose également sur l'existence d'une paire de clés pour chaque participant : une clé privée et une clé publique. La seule chose à retenir est que les deux clés sont liées.

Notons k_{prA} la clé privée d'Alice et k_{puA} sa clé publique.

Etant donné un message m , on a :

$$k_{prA}(k_{puA}(m)) = k_{puA}(k_{prA}(m)) = m$$

Cela signifie que l'on peut chiffrer un message m avec une clé publique et le déchiffrer avec la clé privée associée, mais on peut aussi chiffrer avec la clé privée, et déchiffrer avec la clé publique associée. Les clés k_{prA} et k_{puA} ont donc des rôles interchangeables : l'une des deux clés permet de défaire le chiffrement effectué par l'autre clé.

Les propriétés mathématiques mises en jeux font aussi qu'il est :

- impossible, en connaissant la clé publique k_{puA} , de deviner la clé privée k_{prA} et, réciproquement, si on connaît k_{prA} , il est impossible d'en déduire k_{puA} ;
- impossible, en connaissant uniquement les messages chiffrés $k_{puA}(m)$ ou $k_{prA}(m)$ de deviner le message m .

Une fois ces propriétés établies, si Bob souhaite envoyer un message secret à Alice, les deux protagonistes procèdent comme décrit dans l'illustration page suivante.

La robustesse du système RSA repose sur la difficulté de factoriser un très grand entier n en un produit de deux nombres premiers p et q . Deux événements pourraient faire s'écrouler la sécurité du RSA :

- la découverte d'un algorithme efficace de factorisation, capable de tourner sur les ordinateurs actuels. Cette annonce est régulièrement faite, et tout aussi régulièrement contredite par la communauté scientifique ;
- l'avènement d'ordinateurs quantiques, dont la vitesse d'exécution permettrait une factorisation rapide (il est à noter que l'algorithme de factorisation destiné à tourner sur un ordinateur quantique existe déjà : il s'agit de l'algorithme de Schor).

Pour aller plus loin sur le chiffrement RSA :

Consulter la vidéo « hh » extraite de la chaîne de Mickaël Dupont, enseignant-chercheur à l'USMB Savoie-Mont-Blanc

Lien : <https://www.youtube.com/watch?v=j2tOQkwLndk>

Sécurisation des communications

Un chiffrement asymétrique repose sur l'existence d'une clé publique, appelée à être distribuée largement, et d'une clé privée, qui ne quitte jamais son propriétaire.

Étape 1

Alice fabrique sa clé privée et sa clé publique.



Étape 2

Alice communique à Bob sa clé publique.



Étape 3

Bob chiffre son message avec la clé publique d'Alice.

Une fois son message chiffré, Bob lui-même ne peut pas revenir en arrière et déchiffrer son propre message.



Étape 4

Grâce à sa clé privée, Alice peut ouvrir le message qui lui est adressé.



Source : https://glassus.github.io/terminale_nsi/T5_Architecture_materielle/5.4_Cryptographie/cours/

Sécurisation des communications

IV – Le protocole HTTPS

Les systèmes de chiffrement asymétriques possèdent un inconvénient de taille : le chiffrement et le déchiffrement nécessitent des calculs très coûteux. Cette complexité fait qu'il n'est pas possible de les utiliser pour chiffrer directement de gros volumes de données ou des flux de communication (ex : visioconférence avec envoi de vidéo et de son).

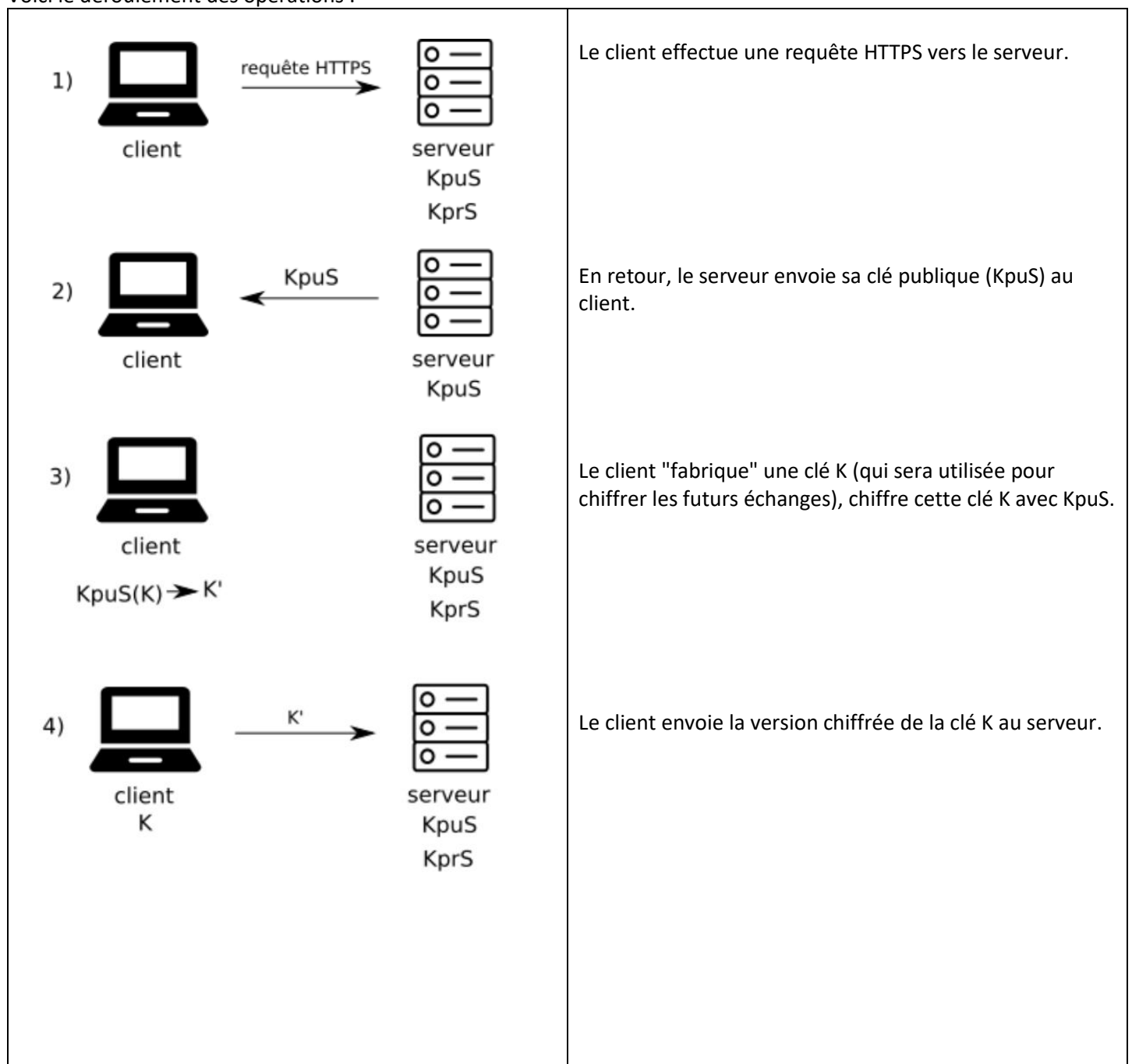
1) Le principe du protocole HTTPS

Le protocole HTTPS (*HyperText Transfer Protocole Secure*) est un exemple d'utilisation conjointe d'un chiffrement symétrique et d'un chiffrement asymétrique. Le but du protocole HTTPS n'est pas de recréer un nouveau protocole, mais juste de rendre HTTP plus sûr en chiffrant les données qui circulent entre le client et le serveur.

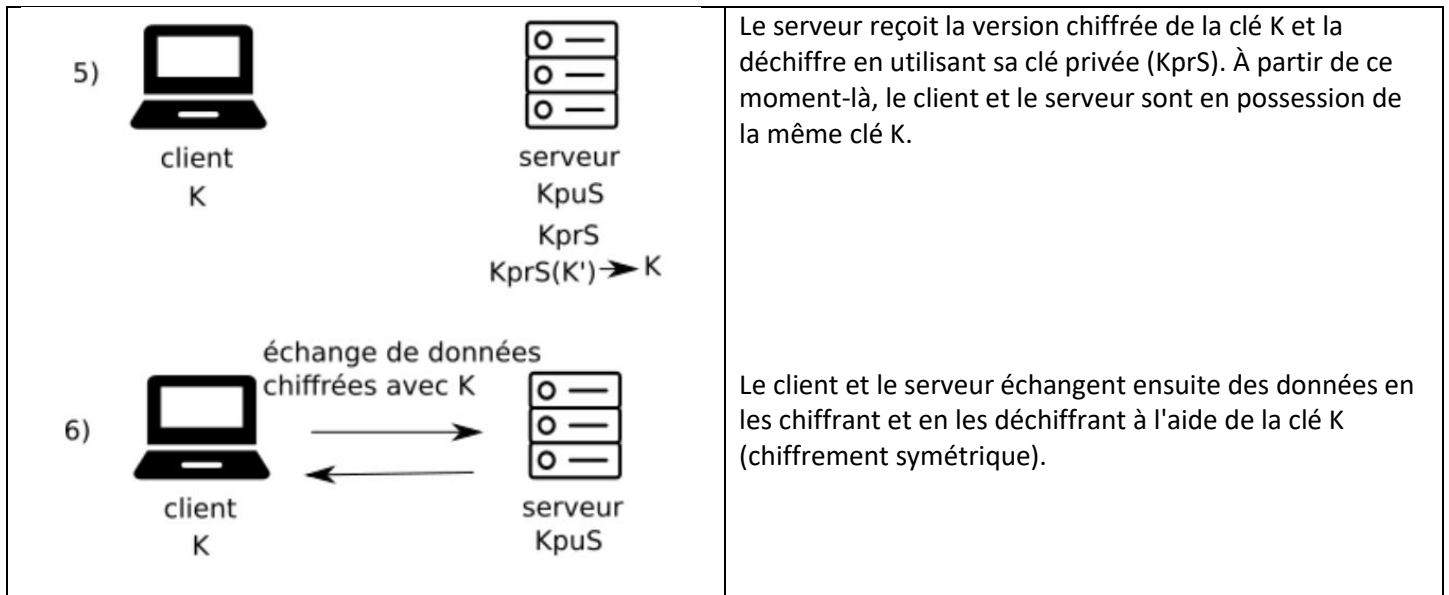
Ces communications vont être chiffrées grâce à une clé symétrique.

Problème : comment échanger cette clé entre le client et le serveur ?

Voici le déroulement des opérations :



Sécurisation des communications



Il faut donc bien comprendre qu'il y a deux étapes de chiffrement :

- 1) **HTTPS utilise un chiffrement asymétrique pour établir une connexion et échanger une clé symétrique.**
- 2) **Ensuite, les échanges utilisent un chiffrement symétrique.**

L'intérêt de ce système est que le chiffrement symétrique est beaucoup moins gourmand en ressources.

Une fois la connexion établie, le protocole HTTPS consomme des ressources CPU assez raisonnables (notons que l'établissement de la connexion est très consommatrice en ressources).

2) L'attaque de l'homme du milieu

Le problème du chiffrement par clé publique est qu'il faut parfois vérifier que la clé publique est bien celle de celui qu'on croit être l'émetteur...

Pour comprendre : cf. la vidéo de cookie connecté de 5:06 à 6:30.

[Comprendre le chiffrement SSL / TLS avec des emojis \(et le HTTPS\)](#)

Le chiffrement des communications permet de se protéger contre l'espionnage des communications, c'est-à-dire l'observation *passive* par un tiers des messages envoyés entre deux participants.

Considérons maintenant le scénario suivant : supposons qu'Alice soit cliente de l'établissement bancaire BobBanque. Les communications entre Alice et BobBanque étant confidentielles, elles doivent évidemment être chiffrées, mais cela n'est pas suffisant.

Eve (la malveillante) peut procéder à l'attaque suivante : elle envoie un e-mail au format HTML à Alice, lui demandant de se connecter d'urgence sur le site de sa banque.

Le contenu d'un tel e-mail pourrait être :

```
<html>
...
  <body>
    Veuillez vous connecter d'urgence au site
    <a href="http://eve.com"> BobBanque.com</a>.
  </body>
</html>
```

Ce dernier s'affichera de la façon suivante :

Veuillez vous connecter d'urgence au site **BobBanque.com**

Sécurisation des communications

Si Alice n'est pas méfiante, et qu'Eve utilise en plus d'autres techniques (comme de reproduire exactement la page d'accueil du site BasileBanque.com), alors Alice ne se rendra pas compte qu'elle est connectée au mauvais site. Eve pourra donc recevoir tous les messages envoyés par Alice. Elle pourra aussi les retransmettre au véritable site BobBanque.com, en se faisant passer pour Alice. Eve sera donc dans une position d'intermédiaire entre Alice et Bob. Elle pourra retransmettre passivement les requêtes d'Alice à BobBanque, ou les modifier avant de les retransmettre (par exemple changer le montant et le destinataire d'un virement bancaire), ce que l'on peut illustrer par le schéma suivant :

Alice <-----> Eve <-----> BasileBanque.com

Ici, le chiffrement n'est d'aucune aide. En effet, même si Alice établit un canal sûr entre elle et Eve, et qu'Eve en établit un entre elle-même et BobBanque, l'attaque peut quand même avoir lieu.

On appelle cela une **attaque de l'homme du milieu** (ou **man in the middle attack** en anglais, souvent abrégé en **MITM**).

La faille fondamentale permettant cette attaque est **l'absence d'authentification**.

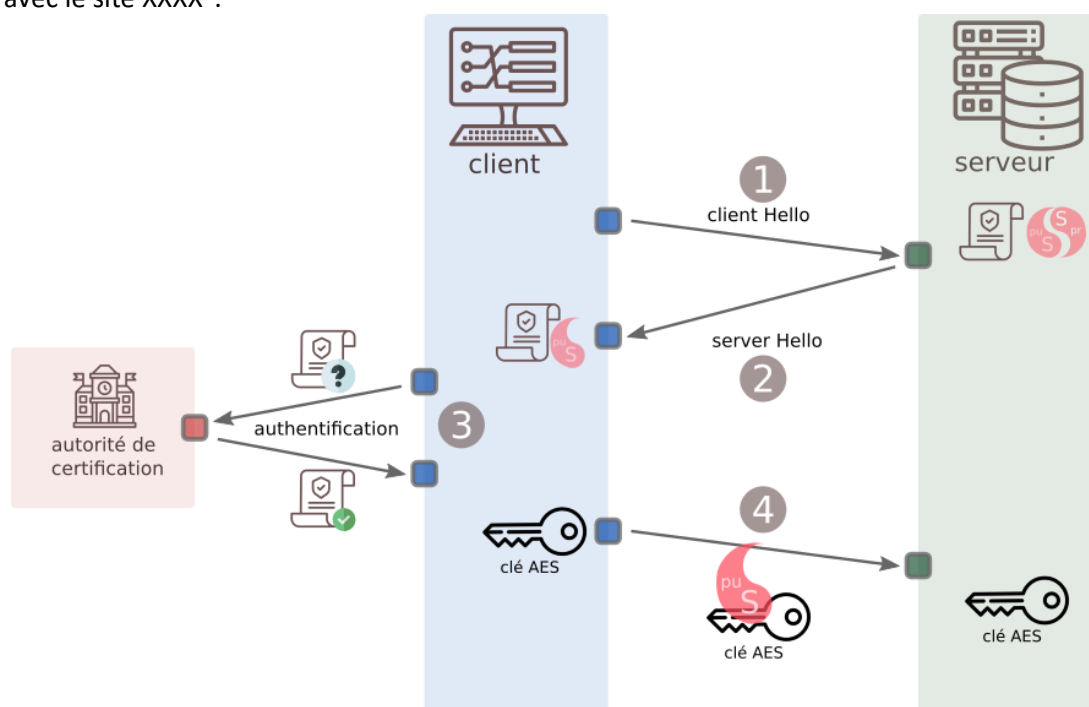
Ici, le serveur d'Eve, qui imite le comportement du serveur de BobBanque, n'a pas eu à *prouver* qu'il était bien une machine appartenant à BobBanque.

2) Certificats d'authentification et tiers de confiance

Pour éviter tout problème, il faut que le serveur puisse justifier de son "identité" ("voici la preuve que je suis bien le site de la banque B et pas un site "pirate"). Pour ce faire, chaque site désirant proposer des transactions HTTPS doit, périodiquement, demander (acheter dans la plupart des cas) un certificat d'authentification (sorte de carte d'identité pour un site Web) auprès d'une autorité habilitée à fournir ce genre de certificats (chaque navigateur web possède une liste des autorités dont il accepte les certificats). Comme dit plus haut, ce certificat permet au site de prouver son "identité" auprès des clients.

Nous n'allons pas entrer dans les détails du fonctionnement de ces certificats, mais vous devez juste savoir que le serveur envoie ce certificat au client en même temps que sa clé publique (cf. schéma ci-dessous et étape 2 du schéma page 8).

En cas d'absence de certificat ou d'envoi de certificat non conforme, le client stoppe immédiatement les échanges avec le serveur. Il peut arriver de temps en temps que le responsable d'un site oublie de renouveler son certificat à temps (dépasse la date d'expiration), dans ce cas, le navigateur web côté client affichera une page de mise en garde avec un message du style "ATTENTION le certificat d'authentification du site XXX a expiré, il serait prudent de ne pas poursuivre vos échanges avec le site XXXX".



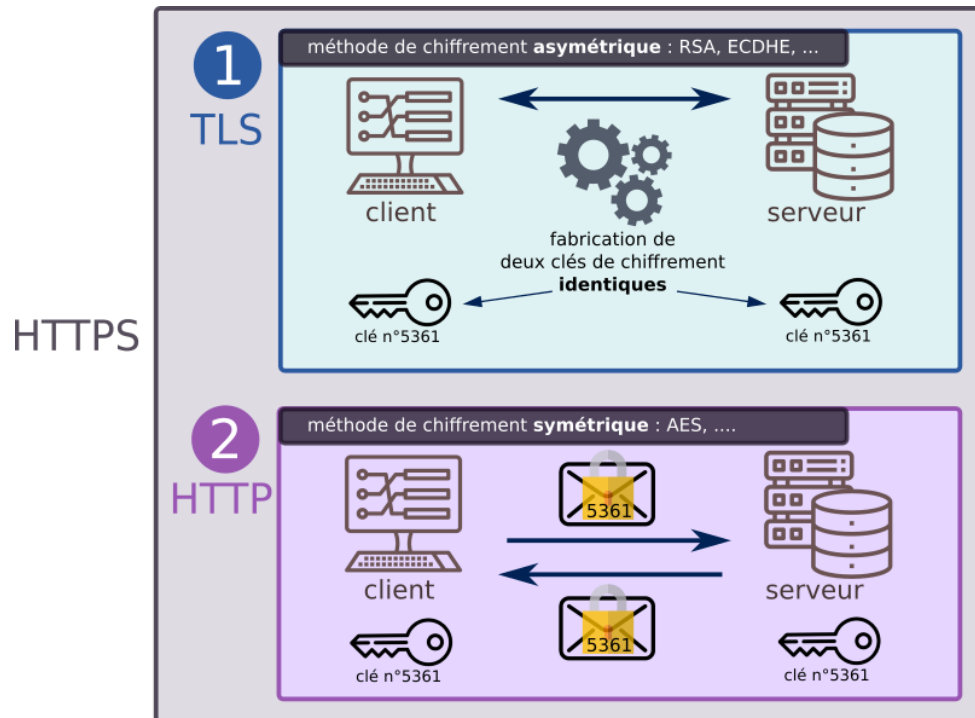
Source : https://glassus.github.io/terminale_nsi/T5_Architecture_materielle/5.4_Cryptographie/cours/

Sécurisation des communications

A retenir :

Le protocole HTTPS est la réunion de deux protocoles :

- le protocole TLS (Transport Layer Security) : ce protocole, basé sur du **chiffrement asymétrique**, va conduire à la génération d'une clé identique chez le client et chez le serveur.
- le (bon vieux) protocole HTTP, mais qui convoiera maintenant des données chiffrées avec la clé générée à l'étape précédente. Les données peuvent toujours être interceptées, mais sont illisibles. Le **chiffrement symétrique** utilisé est actuellement le chiffrement AES.



Source : https://glassus.github.io/terminale_nsi/T5_Architecture_materielle/5.4_Cryptographie/cours/

Sources :

- Numérique et Sciences Informatiques – 24 leçons avec exercices corrigés (T. Balabonski, S. Conchon, JC Filliâtre et K. Nguyen).
- Le site de D. Roche : pixees.fr
- Le site de G. Bareux : http://gisele.bareux.free.fr/NSI1_Term.htm
- Le site de G. Lassus : https://glassus.github.io/terminale_nsi/T5_Architecture_materielle/5.4_Cryptographie/cours/